

Crossover on a Real Neural Architecture Search Space: Structure Does Not Help, and at Scale It Hurts

Vasili Gavrilov*

2026

Abstract

Evolutionary neural architecture search recombines architectures with crossover, but a recurring result is that random search is hard to beat, and it is rarely clear whether crossover contributes anything of its own. We study this directly on NAS-Bench-101, a real, tabulated space of 423,624 cell architectures whose fitness is a lookup, so a run is microseconds and thousands of paired seeds are affordable. Two findings, both reported with paired significance tests. First, on this space *crossover does beat random search*, significantly and at every compute budget. Second, and against intuition, *making the crossover smarter makes it worse*. The NAS-Bench-101 cell is naturally multi-dimensional — a wiring dimension (the adjacency) and a labeling dimension (the operations), plus a graph/path view — and it is tempting to recombine along those dimensions: factor wiring from labeling, align nodes by role before recombining, or transplant whole input-output paths. Every such structured operator underperforms plain per-cell uniform crossover; two of them, at large budget, drop *below random search itself*. We trace this to diversity: the structural priors mismanage the offspring variety that crossover-plus-selection needs — collapsing it into premature convergence (axis, path) or maintaining it without exploiting it (aligned) — while plain uniform crossover balances exploration against exploitation. The lesson restates a known one — random search is a strong NAS baseline, and structural cleverness rarely beats a simple baseline — now with a controlled mechanism on a real space. The fitness oracle, isomorphism-correct by brute-force canonicalization, and every operator are a single dependency-free C program; every number regenerates from one command.

1 Introduction

Neural architecture search (NAS) automates the choice of a network’s shape; methods differ mainly in the search strategy — reinforcement learning [3], gradient relaxation [4], Bayesian optimization, random search, and evolution [6]. Evolutionary NAS mutates and recombines architectures and trains each candidate’s weights by back-propagation [5]. A recurring and sobering result is that plain random search is a strong baseline [2, 7], and within evolutionary methods it is often unclear whether *crossover* earns its place: the field’s leading evolutionary result drops crossover entirely and keeps only mutation with aging [5].

We ask two questions and answer them on a real benchmark. Does crossover beat random search at all? And if the search space has structure — NAS-Bench-101’s cell factors into a wiring dimension and a labeling dimension — does a crossover that *respects that structure* beat one that ignores it? The intuition that it should is strong: a graph’s building blocks are its subgraphs and its wiring, not arbitrary bits, and recombining along the natural axes ought to preserve what makes a parent good. We find the first answer is yes and the second is no — emphatically, and reversing at scale.

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning. Reference implementation: <https://github.com/Anode1/SMBPANN>.

The engine behind this work is a self-modifying back-propagation network whose topology and training hyper-parameters co-evolve [1]; here we use only its recombination logic, and to keep the comparison clean we move fitness onto the NAS-Bench-101 table so no confound of training noise enters.

2 The space and the fitness oracle

A NAS-Bench-101 cell is a directed acyclic graph on up to seven nodes: node 0 is the input, node $n-1$ the output, and the interior nodes carry one of three operations (`conv1x1`, `conv3x3`, `maxpool3x3`). An architecture thus has two orthogonal dimensions, its *wiring* (the adjacency matrix) and its *labeling* (the interior operations), and a third, derived view as a set of input–output paths. The public benchmark records the trained CIFAR-10 accuracy of every one of the 423,624 distinct cells, so fitness is a table lookup and no network is trained during search.

The one subtlety is that the benchmark de-duplicates *isomorphic* graphs: the same abstract cell can be written with its interior nodes permuted, and a recombined child will frequently be a relabeling of a catalogued cell rather than a verbatim match. Looking a child up therefore requires a canonical form. We compute it by brute force: prune the nodes that lie on no input–output path, then take the lexicographically minimum adjacency-plus-labeling encoding over all $\leq 5! = 120$ permutations of the interior nodes. This is exact and dependency-free. A self-test confirms it: over 5,000 catalogued cells, both the verbatim graph and a random interior relabeling of it map to the same accuracy, 5000/5000 each, and the 423,624 table rows collapse to exactly 423,624 distinct canonical keys (no collisions). The same canonicalization supplies the node correspondence used by the alignment operator below.

The data itself is extracted from the official `nasbench_only108.tfrecord` by a small C decoder (TFRecord framing, JSON, and a hand-parsed `ModelMetrics` protobuf); best test accuracy 94.32% and best validation 95.06% reproduce the published optima.

3 Operators

Every individual is a seven-node cell in upper-triangular form (input 0, output 6). All arms share the same population loop, the same light mutation (flip one edge or re-roll one interior op), the same elitism, and the same matched query budget; *the arms differ only in the crossover operator*, so any difference is attributable to it. Selection is on validation accuracy and we report test accuracy, so nothing overfits the reported metric. We compare four crossovers against a random-search control:

- **flat** — per-cell uniform crossover: each adjacency bit and each op is copied from a uniformly chosen parent. It ignores the graph structure entirely; it is the maximum-diversity baseline (and the “competing conventions” strawman, since node k in one parent need not play the same role as node k in the other).
- **axis** — dimension-factored: the whole wiring is taken from one parent and the whole labeling from the other. This recombines the two dimensions as units.
- **aligned** — role-aligned: match the interior nodes of the second parent to the first by role (operation and in/out degree), bringing it into the first parent’s coordinate frame, then recombine *node modules* — each node’s whole incoming-edge set and its op come together from one aligned parent. This removes the competing-conventions problem before recombining.
- **path** — subgraph transplant: a cell is a set of input–output paths; recombine whole paths from both parents, each path carrying its own parent’s ops, added greedily under the nine-edge cap. This recombines along the graph dimension.

Table 1: NAS-Bench-101, population 50, elite 12, 600 paired seeds. Mean best test-accuracy (%), and Wilcoxon signed-rank z against random search (positive = beats random). Flat crossover wins at every budget; *axis* and *path* drop below random at scale and aligned’s edge decays to zero. Every arm shares each seed with the control.

budget	random	flat (z)	axis (z)	aligned (z)	path (z)
500	93.639	94.009 (+19.0)	93.792 (+11.4)	93.909 (+16.2)	93.922 (+17.6)
2000	93.743	94.082 (+17.8)	93.852 (+7.7)	94.001 (+15.0)	93.930 (+13.0)
8000	93.940	94.094 (+12.3)	93.867 (−5.2)	94.032 (+7.4)	93.933 (−0.6)
20000	94.037	94.096 (+6.2)	93.873 (−10.9)	94.042 (+0.9)	93.934 (−8.0)

Matched compute and pairing. Each arm may query the oracle a fixed number of times; the random control spends the same number of queries on independent random cells; a child that prunes to an invalid or over-budget graph is a spent query scored zero, so an operator that wastes queries on invalid graphs is penalized honestly. Crucially, in each trial the control and all GA arms are given the *same* seed, so they start from an identical initial population; a trial is thus a paired observation, and we report both the paired win/loss counts and the Wilcoxon signed-rank z of (arm − baseline).

4 Results

4.1 Crossover beats random search

At a small population (20, elite 5, 800 seeds), every crossover arm beats random search significantly at every budget (Wilcoxon z from 2 to 14; win rates well above one half). Unlike the tiny six-gene NAS-Bench-201 cell, where crossover’s edge over random is a fraction of a percent and vanishes at any generous budget, the larger NAS-Bench-101 space gives recombination something real to exploit. So the first question is settled in the affirmative: crossover helps here.

4.2 Structure does not help — and at scale it hurts

The second question is the substance. Table 1 scales the population to 50 (elite 12) and the budget to 20,000 evaluations, 600 paired seeds. The ordering is stark and monotone. Plain **flat** crossover is the best at *every* budget. The structured operators — the ones that respect the space’s dimensions — are worse, and worsen as compute grows: the dimension-factored **axis** operator falls *below random search* at the two larger budgets (a significantly negative z), and the aligned operator’s advantage over random decays to nothing.

The trends are clean and monotone across the four budgets. Flat’s advantage over random shrinks as random saturates (its z falls 19 → 18 → 12 → 6) but never disappears; it beats every structured operator head-to-head at every budget. Axis, which factors the two dimensions into large indivisible units, converges prematurely and its z against random crosses from +11 to −11; by 20,000 evaluations it loses to random search 176/399. The path operator behaves the same way and more starkly: its mean best accuracy barely moves across a fortyfold increase in budget (93.922 → 93.930 → 93.933 → 93.934) — it has essentially converged by 500 evaluations and then stops — so random search sails past it and it too ends significantly *below* random ($z = -8.0$). Only the aligned operator, which mixes at the intermediate granularity of node modules, holds on: its edge over random decays from $z = 16$ to $z = 0.9$, exactly to the break-even point. So of the four crossovers, the one that ignores structure wins at every budget, the two most structured ones (axis, path) fall below random search at scale, and the intermediate one ends in a tie with it.

Table 2: Population sweep at fixed budget 2000, 600 paired seeds (elite = pop/4; the random control is identical throughout, mean 93.743). Mean best test-accuracy (%) and Wilcoxon z vs random. Crossover’s benefit is created by diversity: at pop 8 every arm loses to random; the gain rises monotonically with population.

pop	flat (z)	axis (z)	aligned (z)	path (z)
8	93.673 (−4.5)	93.652 (−5.7)	93.673 (−4.2)	93.526 (−12.6)
16	93.791 (+3.5)	93.715 (−1.8)	93.756 (+1.1)	93.706 (−2.7)
32	93.981 (+14.6)	93.823 (+5.9)	93.898 (+10.5)	93.840 (+7.3)
64	94.122 (+19.3)	93.917 (+11.6)	94.064 (+17.2)	93.957 (+14.2)

The obvious escape — that the structured operators fail because they produce more invalid graphs — is closed by measurement: axis, aligned, and path in fact produce *more* valid offspring (95–97%) than flat (90–96%), so flat wins on genuine search quality, not by dodging invalid graphs.

4.3 The mechanism is diversity

Two lines of evidence bear on the cause, and together they refine it from raw diversity to *productive* diversity. First, measuring the final population’s genotypic spread (mean pairwise gene distance over the 21 edges and 5 ops) exposes two distinct failure modes. The two operators that end below random search, axis and path, are exactly the two that *collapse* that spread (final diversity 1.8 and 1.3 against flat’s 2.1 at 20,000 evaluations): they converge prematurely, which is precisely the plateau their accuracy curves show. But diversity alone is not the whole story — the aligned operator maintains the *most* spread of all (3.1, above flat) and still only ties random, because its structural recombination keeps the population varied without concentrating it on good cells. Flat holds an intermediate, productive spread and wins. So the structured operators fail in one of two ways — by collapsing diversity (axis, path) or by spreading it unproductively (aligned) — and only the structure-blind operator balances exploration against exploitation. (We report this rather than the cleaner “diversity ranks the operators” story we first expected; the measurement does not support that.)

Second, and independently, varying the *population* size at fixed budget sweeps the diversity *available* to crossover, and the whole comparison moves with it (Table 2). At a tiny population of eight, crossover has too little variety to work with and *every* arm, flat included, loses to random search ($z < 0$). As the population grows the crossover gain climbs monotonically for every operator, until at population 64 all four beat random decisively. Crossover on this space is not intrinsically better or worse than random search; it is better exactly when it has enough diversity to draw on and an operator that turns diversity into good offspring rather than dissipating or collapsing it.

So crossover on this space works because fine-grained mixing plus selection keeps proposing varied candidates near good regions; every structural prior we add trades that variety for a coherence the space does not reward, and the more compute we spend, the more the low-diversity operators converge early and are overtaken by random search, which never stops exploring.

5 Relation to prior work

The framing is not new and we do not claim it is. Random search as a strong NAS baseline is [2, 7]; mutation-only evolutionary NAS that deliberately omits crossover is [5]; the difficulty of meaningful crossover on graph genomes — the competing-conventions problem that our aligned operator is designed to remove — is the motivation of NEAT’s historical markings [8]. NAS-

Bench-101 [9] is the tabulated space that makes a study at this seed count possible. Our contribution is not a method but a clean, controlled, reproducible *negative* result: on a real multi-dimensional NAS space, four different structure-respecting crossovers all lose to plain uniform crossover, two of them to random search, with a diversity mechanism that explains why.

6 Discussion and conclusion

On NAS-Bench-101 crossover beats random search — a genuine positive, larger and cleaner than the same comparison on the smaller NAS-Bench-201 cell. But the appealing idea that a crossover should recombine along the search space’s natural dimensions does not survive contact with the data: factoring the wiring and labeling dimensions, aligning nodes by role, and transplanting whole paths all underperform a structure-blind uniform crossover, and the more structured the operator the sooner it is overtaken by random search as the budget grows. The cause is diversity, not validity: structural priors narrow the offspring distribution that crossover-plus-selection depends on. **Limitations:** results are on one tabulated cell space with a table-lookup fitness (no training-time confounds, but also no transfer to larger or hierarchical spaces); the aligned operator is one reasonable alignment and a better one may exist, though the burden is now on exhibiting one that beats uniform crossover, which four natural attempts do not. The natural next step is a larger, hierarchical space where recombination has more room to matter.

Reproducibility

Every number regenerates from [10] with one command. The fitness table is extracted from the official benchmark by `nb101_extract.c` (`make nb101_extract`); the search, oracle, and all operators are `validation/nb101.c` (`make nb101`), configurable by the environment (`POP`, `ELITE`, `SEEDS`, `BUDGETS`) and self-testing its oracle with `-selftest`. Both build clean under `-std=c99` `-pedantic` and run clean under AddressSanitizer and UBSan. A fixed xorshift generator makes every run reproducible.

References

- [1] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks*. Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). <https://doi.org/10.5281/zenodo.20450526>
- [2] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.
- [3] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.
- [4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.
- [5] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.
- [6] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.
- [7] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.
- [8] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.
- [9] C. Ying, A. Klein, E. Real, E. Christiansen, K. Murphy, F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML* 2019.
- [10] SMBPANN reference implementation. <https://github.com/Anode1/SMBPANN>.